
C PROGRAMMING LAB REPORT

Menu-Driven Programs

Question	Topic
Question 1	Swapping two numbers
Question 2	Sum of natural numbers and sum of squares
Question 3	Simple Interest and Compound Interest

Name: Reyansh Pandit

Batch:16

SAP ID :590033602

QUESTION 1 — SWAPPING TWO NUMBERS

Problem Statement: Write a menu-driven C program to input two numbers and swap them using a third variable and without using a third variable. Also display the values before and after swapping.

1. Understanding the Problem Statement

The program accepts two numbers and displays their values before swapping. The user chooses either swapping with a third variable or swapping without a third variable. In the first method, a temporary variable stores one value. In the second method, arithmetic operations are used.

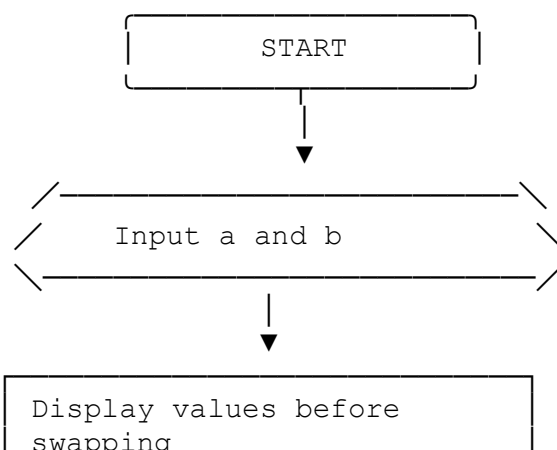
2. Standard Test Case Table

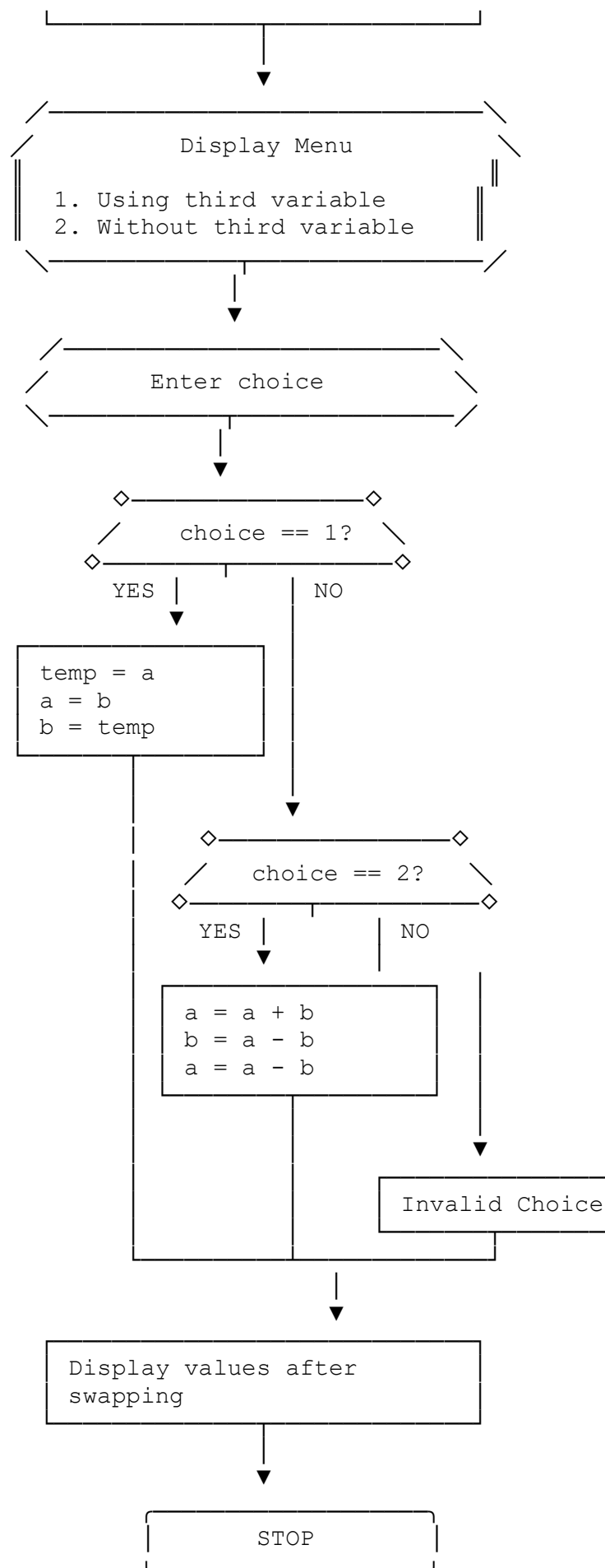
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	a=10, b=20, Choice=1	Before 10,20; After 20,10	Before 10,20; After 20,10	Pass
2	a=10, b=20, Choice=2	Before 10,20; After 20,10	Before 10,20; After 20,10	Pass
3	a=5, b=15, Choice=1	Before 5,15; After 15,5	Before 5,15; After 15,5	Pass
4	a=-5, b=10, Choice=2	Before -5,10; After 10,-5	Before -5,10; After 10,-5	Pass
5	a=25, b=25, Choice=1	Before 25,25; After 25,25	Before 25,25; After 25,25	Pass
6	a=10, b=20, Choice=3	Invalid choice	Invalid choice	Pass

3. Algorithm

1. Start.
2. Declare a, b, temp, and choice.
3. Input two numbers a and b.
4. Display the values before swapping.
5. Display the menu and input the user's choice.
6. If choice is 1, use temp to swap a and b.
7. If choice is 2, use $a=a+b$, $b=a-b$, $a=a-b$ to swap.
8. If the choice is invalid, display Invalid Choice.
9. Display the values after swapping when a valid choice is selected.
10. Stop.

4. Flowchart





5. Pseudocode

```
BEGIN
INPUT a, b
DISPLAY a, b before swapping
DISPLAY menu
INPUT choice
IF choice == 1 THEN
temp = a
a = b
b = temp
END IF
IF choice == 2 THEN
a = a + b
b = a - b
a = a - b
END IF
IF choice is not 1 or 2 THEN
DISPLAY "Invalid Choice"
ELSE
DISPLAY a, b after swapping
END IF
END
```

6. C Program

```
#include <stdio.h>

int main()
{
int a, b, temp, choice;

printf("Enter two numbers: ");
scanf("%d %d", &a, &b);

printf("\nBefore swapping: a = %d, b = %d\n", a, b);

printf("\n--- Swapping Menu ---\n");
printf("1. Swap using third\nvariable\n"); printf("2. Swap\nwithout third variable\n");
printf("Enter your choice: ");
scanf("%d", &choice);

if (choice == 1)
{
temp = a;
a = b;
b = temp;
printf("\nAfter swapping: a = %d, b = %d\n", a, b);
}

if (choice == 2)
{
a = a + b;
b = a - b;
a = a - b;
printf("\nAfter swapping: a = %d, b = %d\n", a, b);
}

if (choice != 1 && choice != 2)
printf("\nInvalid choice!\n");

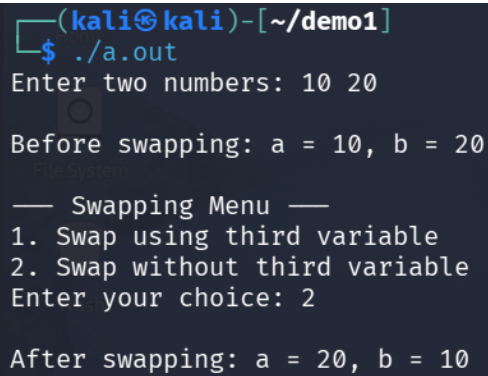
return 0;
}
```

7. Compilation and Execution

```
cc swap.c
./a.out
```

Sample execution:

```
Enter two numbers: 10 20
Before swapping: a = 10, b = 20
--- Swapping Menu ---
1. Swap using third variable
2. Swap without third variable
Enter your choice: 2
After swapping: a = 20, b = 10
```

A terminal window screenshot with a dark background. The prompt is '(kali㉿kali)-[~/demo1]'. The user enters './a.out'. The program outputs 'Enter two numbers: 10 20', 'Before swapping: a = 10, b = 20', a 'Swapping Menu' with two options, and 'Enter your choice: 2'. Finally, it outputs 'After swapping: a = 20, b = 10'.

```
(kali㉿kali)-[~/demo1]
$ ./a.out
Enter two numbers: 10 20

Before swapping: a = 10, b = 20

— Swapping Menu —
1. Swap using third variable
2. Swap without third variable
Enter your choice: 2

After swapping: a = 20, b = 10
```

8. Evaluation

The program correctly swaps the two numbers for both methods and correctly rejects an invalid menu choice. All prepared test cases pass.

QUESTION 2 — SUM OF NATURAL NUMBERS

Problem Statement: Write a menu-driven C program to find the sum of the first n natural numbers and the sum of squares of the first n natural numbers.

1. Understanding the Problem Statement

The program accepts a positive integer n and provides two choices: calculate the sum of the first n natural numbers, or calculate the sum of their squares. A FOR loop is used for the calculations.

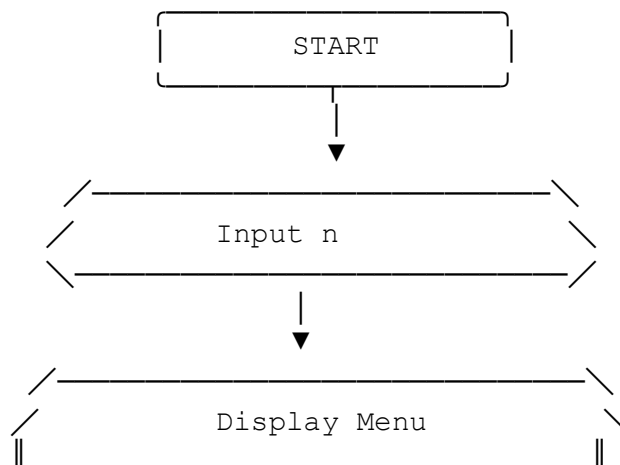
2. Standard Test Case Table

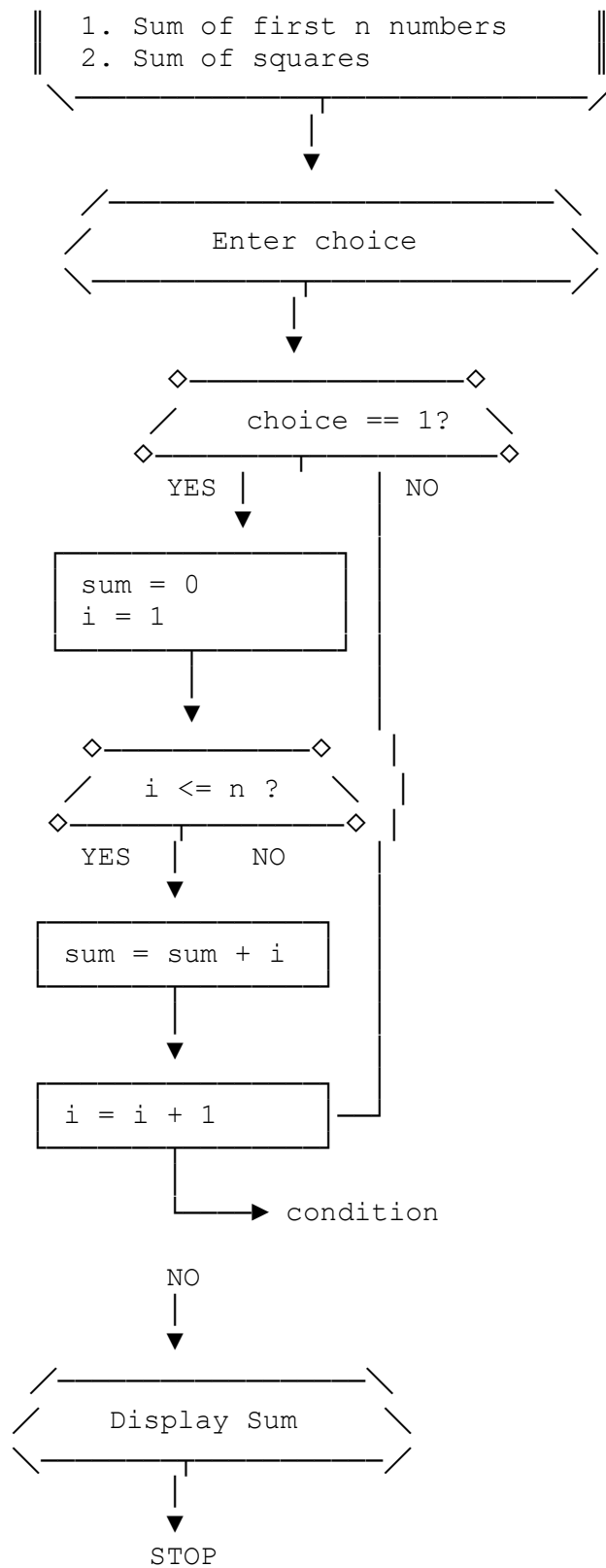
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	n=5, Choice=1	Sum = 15	Sum = 15	Pass
2	n=5, Choice=2	Square sum = 55	Square sum = 55	Pass
3	n=10, Choice=1	Sum = 55	Sum = 55	Pass
4	n=10, Choice=2	Square sum = 385	Square sum = 385	Pass
5	n=1, Choice=1	Sum = 1	Sum = 1	Pass
6	n=1, Choice=2	Square sum = 1	Square sum = 1	Pass
7	n=5, Choice=3	Invalid choice	Invalid choice	Pass

3. Algorithm

1. Start.
2. Declare n, choice, i, sum, and square_sum.
3. Input the positive integer n.
4. Display the menu and input the choice.
5. For choice 1, initialize sum=0 and add i from 1 to n using a FOR loop.
6. For choice 2, initialize square_sum=0 and add $i \times i$ from 1 to n using a FOR loop.
7. For an invalid choice, display Invalid Choice.
8. Stop.

4. Flowchart





[For Choice 2, use the same loop with:]

```
square_sum = square_sum + i * i
```

5. Pseudocode

```
BEGIN
INPUT n
DISPLAY "1. Sum of first n natural numbers"
DISPLAY "2. Sum of squares of first n natural numbers"
INPUT choice

IF choice == 1 THEN
sum = 0
FOR i = 1 TO n
sum = sum + i
END FOR
DISPLAY sum
END IF

IF choice == 2 THEN
square_sum = 0
FOR i = 1 TO n
square_sum = square_sum + i * i
END FOR
DISPLAY square_sum
END IF

IF choice is not 1 or 2 THEN
DISPLAY "Invalid Choice"
END IF
END
```

6. C Program

```
#include <stdio.h>

int main()
{
int n, choice, i;
long long sum = 0, square_sum = 0;
printf("Enter a positive integer n: ");
scanf("%d", &n);

printf("\n--- Menu ---\n");
printf("1. Sum of first n natural numbers\n");
printf("2. Sum of squares of first n natural numbers\n");
printf("Enter your choice: ");
scanf("%d", &choice);

if (choice == 1)
{
for (i = 1; i <= n; i++)
sum = sum + i;

printf("\nSum of first %d natural numbers = %lld\n", n, sum);
}

if (choice == 2)
{
for (i = 1; i <= n; i++)
square_sum = square_sum + (long long)i * i;

printf("\nSum of squares of first %d natural numbers = %lld\n",
n, square_sum);
}

if (choice != 1 && choice != 2)
printf("\nInvalid choice!\n");

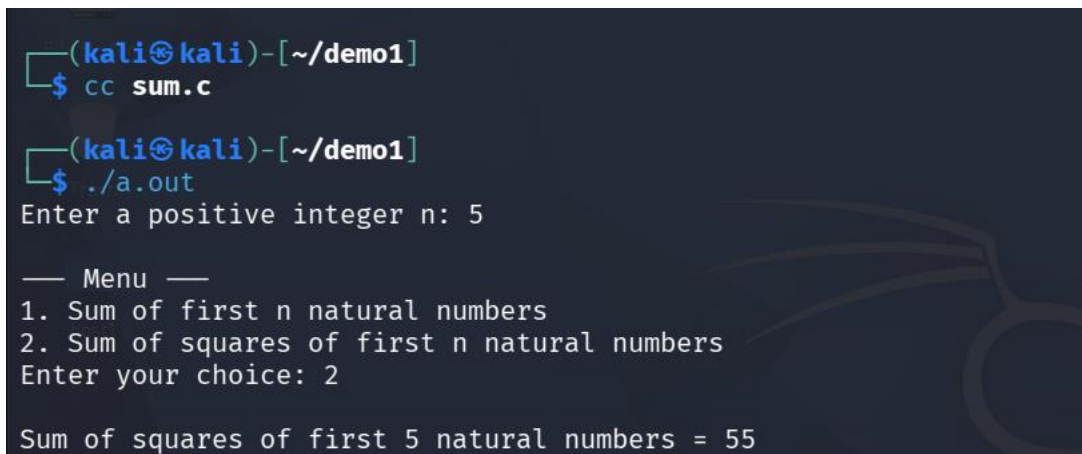
return 0;
}
```

7. Compilation and Execution

```
cc sum.c
./a.out
```

Sample execution:

```
Enter a positive integer n: 5
--- Menu ---
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter your choice: 2
Sum of squares of first 5 natural numbers = 55
```

A screenshot of a terminal window with a dark background. The prompt is (kali@kali)-[~/demo1]. The user enters \$ cc sum.c, followed by \$./a.out. The program then prompts 'Enter a positive integer n: 5'. It displays a menu with two options: '1. Sum of first n natural numbers' and '2. Sum of squares of first n natural numbers'. The user enters '2'. The program outputs 'Sum of squares of first 5 natural numbers = 55'.

8. Evaluation

The FOR loop correctly processes every integer from 1 through n. The program produces the expected sums for all prepared cases and correctly handles an invalid menu choice. All test cases pass.

QUESTION 3 — SIMPLE AND COMPOUND INTEREST

Problem Statement: Write a menu-driven C program to calculate simple interest, compound amount, compound interest, and total payable amount for given principal, rate, and time.

1. Understanding the Problem Statement

The program accepts principal (P), rate of interest (R), and time (T). The menu allows the user to calculate simple interest, compound amount, compound interest, or total payable amount.

Formulas

Quantity	Formula
Simple Interest (SI)	$SI = (P \times R \times T) / 100$
Compound Amount (A)	$A = P \times (1 + R/100)^T$
Compound Interest (CI)	$CI = A - P$
Total Payable Amount	$Total = P + CI = A$

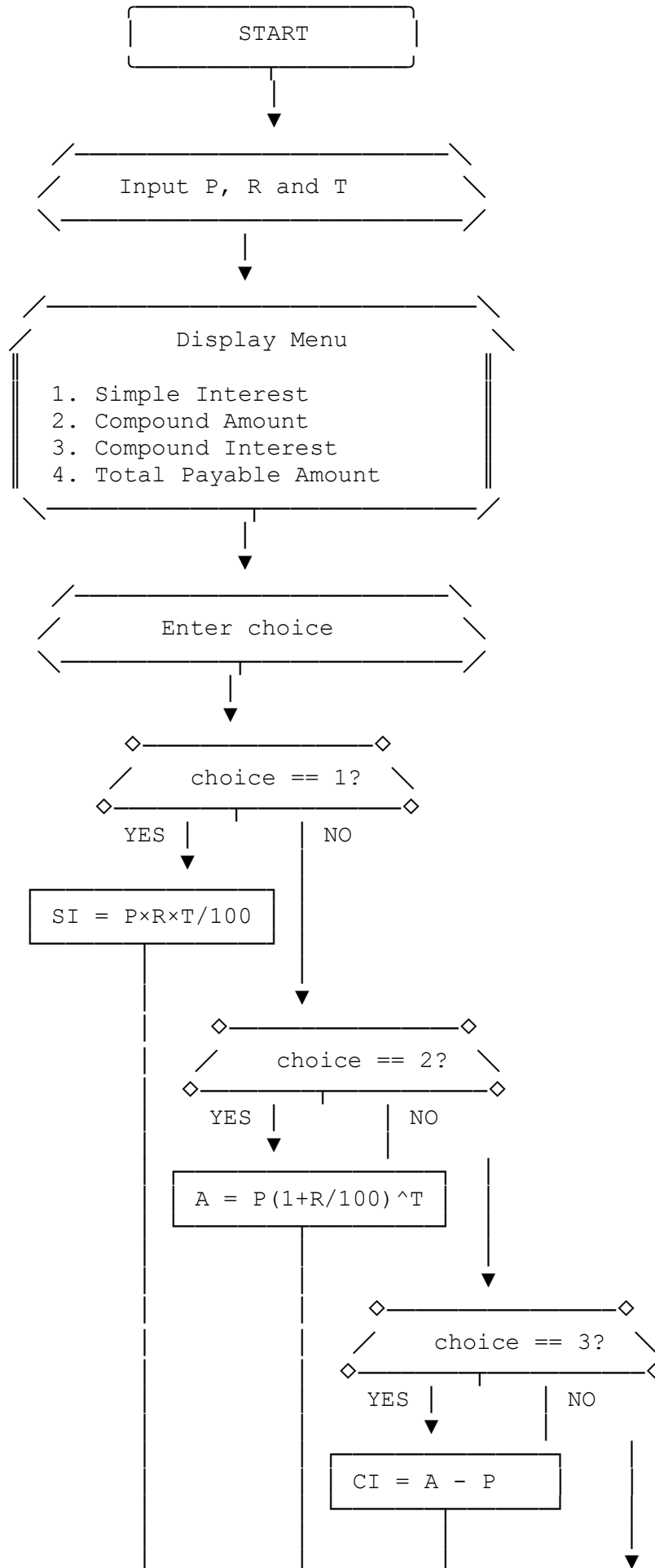
2. Standard Test Case Table

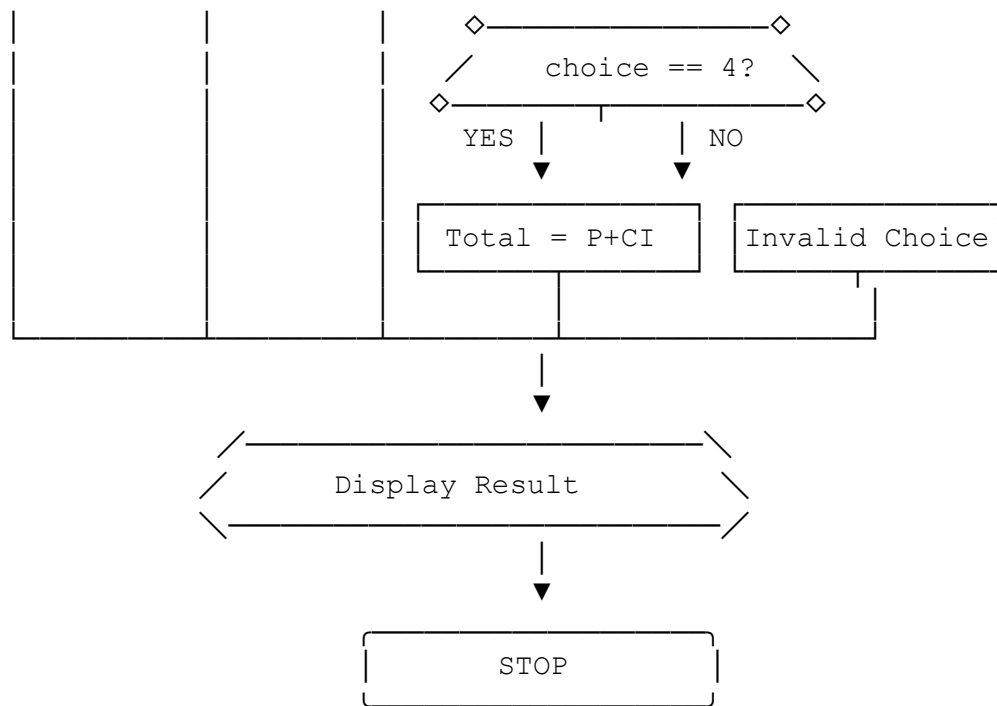
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	P=10000,R=10,T=2,C=1	SI = 2000.00	SI = 2000.00	Pass
2	P=10000,R=10,T=2,C=2	Amount = 12100.00	Amount = 12100.00	Pass
3	P=10000,R=10,T=2,C=3	CI = 2100.00	CI = 2100.00	Pass
4	P=10000,R=10,T=2,C=4	Total = 12100.00	Total = 12100.00	Pass
5	P=5000,R=5,T=1,C=1	SI = 250.00	SI = 250.00	Pass
6	P=5000,R=5,T=1,C=3	CI = 250.00	CI = 250.00	Pass
7	P=10000,R=10,T=2,C=5	Invalid choice	Invalid choice	Pass

3. Algorithm

1. Start.
2. Declare P, R, T, SI, amount, CI, total, and choice.
3. Input principal, rate, and time.
4. Display the four calculation choices and input the user's choice.
5. Calculate $SI = P \times R \times T / 100$.
6. Calculate $A = P \times (1 + R/100)^T$.
7. Calculate $CI = A - P$.
8. Calculate $Total = P + CI$.
9. Display the result corresponding to the selected choice.
10. For an invalid choice, display Invalid Choice.
11. Stop.

4. Flowchart





5. Pseudocode

```

BEGIN
INPUT P, R, T
DISPLAY "1. Simple Interest"
DISPLAY "2. Compound Amount"
DISPLAY "3. Compound Interest"
DISPLAY "4. Total Payable Amount"
INPUT choice

SI = (P * R * T) / 100
A = P * (1 + R / 100)^T
CI = A - P
Total = P + CI

IF choice == 1 THEN DISPLAY SI
IF choice == 2 THEN DISPLAY A
IF choice == 3 THEN DISPLAY CI
IF choice == 4 THEN DISPLAY Total

IF choice is not 1, 2, 3 or 4 THEN
DISPLAY "Invalid Choice"
END IF
END

```

6. C Program

```

#include <stdio.h>
#include <math.h>

int main()
{
double principal, rate, time;
double simple_interest, compound_amount;
double compound_interest, total_payable;
int choice;

printf("Enter principal amount: ");

```

```

scanf("%lf", &principal);
printf("Enter rate of interest: ");
scanf("%lf", &rate);
printf("Enter time in years: ");
scanf("%lf", &time);

printf("\n--- Interest Calculation Menu ---\n");
printf("1. Simple Interest\n");
printf("2. Compound Amount\n");
printf("3. Compound Interest\n");
printf("4. Total Payable Amount\n");
printf("Enter your choice: ");
scanf("%d", &choice);

simple_interest = (principal * rate * time) / 100.0;
compound_amount = principal * pow((1 + rate / 100.0), time);
compound_interest = compound_amount - principal;
total_payable = principal + compound_interest;

if (choice == 1)
printf("\nSimple Interest = %.2f\n", simple_interest);
if (choice == 2)
printf("\nCompound Amount = %.2f\n", compound_amount);
if (choice == 3)
printf("\nCompound Interest = %.2f\n", compound_interest);
if (choice == 4)
printf("\nTotal Payable Amount = %.2f\n", total_payable);
if (choice < 1 || choice > 4)
printf("\nInvalid choice!\n");

return 0;
}

```

7. Compilation and Execution

```

cc interest.c
./a.out

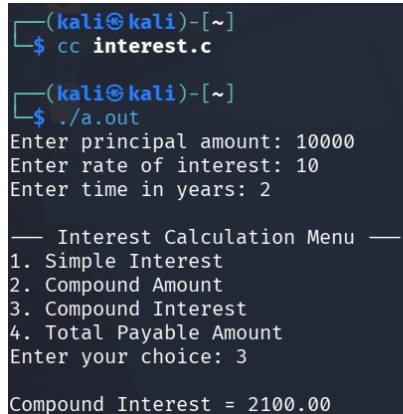
```

Sample execution:

```

Enter principal amount: 10000
Enter rate of interest: 10
Enter time in years: 2
--- Interest Calculation Menu ---
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 3
Compound Interest = 2100.00

```



```

(kali㉿kali)-[~]
$ cc interest.c

(kali㉿kali)-[~]
$ ./a.out
Enter principal amount: 10000
Enter rate of interest: 10
Enter time in years: 2

--- Interest Calculation Menu ---
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount
Enter your choice: 3

Compound Interest = 2100.00

```

8. Evaluation

The program correctly applies the simple-interest and compound-interest formulas and displays the result selected from the menu. The invalid-choice case is also handled. All prepared test cases pass.

Overall Result

The three menu-driven C programs were successfully designed using algorithms, flowcharts, and pseudocode, implemented in C, compiled, executed, and evaluated against the prepared test cases. All listed test cases produced the expected results.